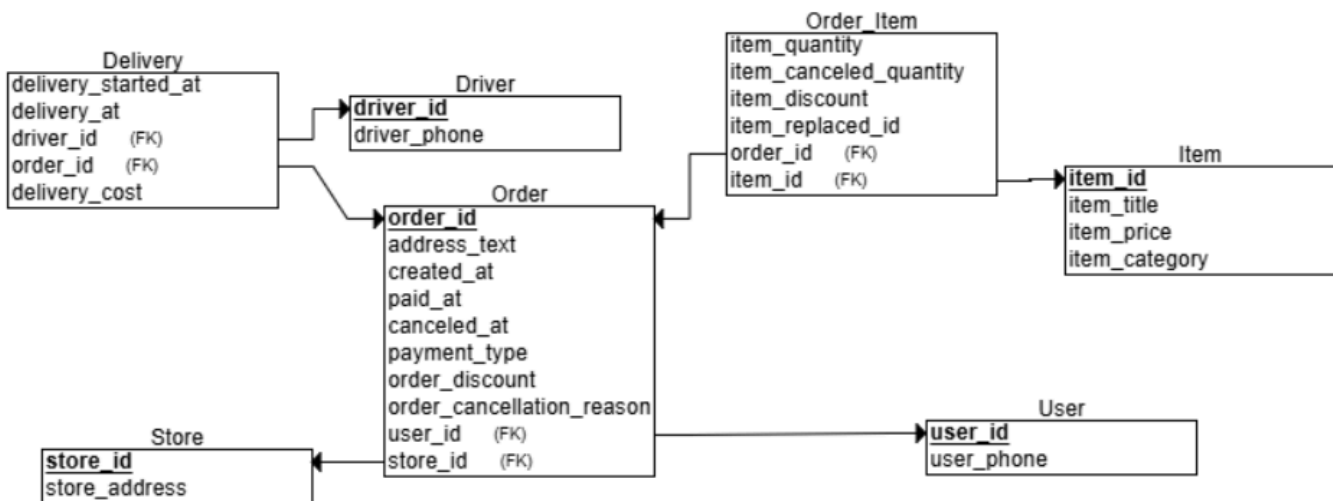


Final Project

Team Number 10

1. Mikhail Sakhno
2. Nabil Atta
3. Frank Carrasco
4. Waleed Amawy

1. Normalized Relational Schema



It's important to mention that there are some missing parts in the schema. For example, in the order_item table, the item_replaced_id is another Foreign Key to Item(item_id). On the other hand, in the Delivery table could be included an auto-incremental primary key called DeliveryID, or could be set the combination of Driver_id and Order_id as Primary Key. For this scenario, we decided not to include that key.

For this scenario, we have defined 3 schemas for different purposes.

RDV(Raw Data Vault), UDV(Universal Data Vault) and DDV(Dimensional Data Vault)

2. Database Structure

Our data and tables are going to be stored in **Postgres-Main** server and inside **main_db** Database. Inside there, we have defined 3 layers in 3 separate schemas(similar to Medallion Architecture).

a. RDV(Raw Data Vault)

In this schema we have stored the raw data, incoming from the parquet files.

This table schema is calculated dynamically using the Pyspark schema interpreter.

b. UDV(Normalized tables)

In this schema we have stored the normalized data, incoming from the raw table.

The DDL for the existing tables:

```
CREATE SCHEMA IF NOT EXISTS udv;
```

```
CREATE TABLE IF NOT EXISTS udv.users (  
    user_id BIGINT PRIMARY KEY,  
    user_phone VARCHAR(20)  
);
```

```
CREATE TABLE IF NOT EXISTS udv.items (  
    item_id BIGINT PRIMARY KEY,  
    item_title TEXT,  
    item_price NUMERIC(10,2),  
    item_category TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS udv.drivers (  
    driver_id BIGINT PRIMARY KEY,
```

```
    driver_phone VARCHAR(20)
);
```

```
CREATE TABLE IF NOT EXISTS udv.stores (
    store_id BIGINT PRIMARY KEY,
    store_address TEXT
);
```

```
CREATE TABLE IF NOT EXISTS udv.orders (
    order_id BIGINT PRIMARY KEY,
    address_text TEXT,
    created_at TIMESTAMP,
    paid_at TIMESTAMP,
    canceled_at TIMESTAMP,
    payment_type TEXT,
    order_discount NUMERIC(5,2),
    order_cancellation_reason TEXT,
    user_id BIGINT,
    store_id BIGINT,
```

```
CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id)
    REFERENCES udv.users (user_id),
```

```
CONSTRAINT fk_orders_store
    FOREIGN KEY (store_id)
```

```
REFERENCES udv.stores (store_id)
);
```

```
CREATE TABLE IF NOT EXISTS udv.deliveries (
    order_id BIGINT,
    delivery_started_at TIMESTAMP,
    delivered_at TIMESTAMP,
    delivery_cost NUMERIC(10,2),
    driver_id BIGINT,

    CONSTRAINT fk_delivery_order
        FOREIGN KEY (order_id)
        REFERENCES udv.orders (order_id),

    CONSTRAINT fk_delivery_driver
        FOREIGN KEY (driver_id)
        REFERENCES udv.drivers (driver_id)
);
```

```
CREATE TABLE IF NOT EXISTS udv.order_items (
    order_id BIGINT,
    item_id BIGINT,
    item_quantity INT,
    item_canceled_quantity INT,
    item_discount NUMERIC(5,2),
    item_replaced_id BIGINT,
```

```
CONSTRAINT fk_order_items_order  
    FOREIGN KEY (order_id)  
    REFERENCES udv.orders (order_id),
```

```
CONSTRAINT fk_order_items_item  
    FOREIGN KEY (item_id)  
    REFERENCES udv.items (item_id),
```

```
CONSTRAINT fk_order_items_replaced_item  
    FOREIGN KEY (item_replaced_id)  
    REFERENCES udv.items (item_id)
```

```
);
```

c. DDV(DataMarts)

In this schema we have stored the data marts using aggregated data, incoming from Universal Tables.

This table schema is calculated dynamically using the Pyspark schema interpreter.

3. Deployment in Docker

After cloning the repo, and changing the branch to “final_project” branch. Execute the command **docker-compose up --build**.

This is an example of how the deployment looks like in a Windows Environment:

Containers [Give feedback](#)

Container CPU usage 2.43% / 1600% (16 CPUs available) Container memory usage 1.85GB / 7.39GB [Show charts](#)

Search Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Mem	Actions
☐	dataengineering_team10	-	-	-	2.43%	1.85GB / 37.85GE		
☐	postgres-main	e6dd9b556ca6	postgres:11	5433:5432	0%	166.1MB / 7.57GI		
☐	postgres-airflow	ea274238c6d1	postgres:11	5434:5432	0.55%	64.96MB / 7.57GI		
☐	airflow-scheduler-1	a30b57f42f47	dataengine		1.8%	329.4MB / 7.57GI		
☐	airflow-webserver-1	af73c0870448	dataengine	8080:8080	0.06%	1.09GB / 7.57GB		
☐	pgadmin	5b8f3eb5e777	dpage/pg	5050:80	0.02%	214.1MB / 7.57GI		

In case some of the services don't start(mainly for resource reasons), please run it manually.

For deployment in MacOS. First execute the following command, in order to set privileges to the entrypoint.sh file: `chmod +x airflow/entrypoint.sh`

4. Data

a. RDV

i. rdv.orders_raw

localhost

pgAdmin 4

main_db/admin@Postgres Main

Query

```
1 select * from rdv.orders_raw
2 limit 100
```

Data Output

	order_id bigint	user_id bigint	user_phone text	address_text text	created_at timestamp without time zone	paid_at timestamp without time zone	delivery_started_at timestamp without time zone	delivered_at timestamp with time zone
1	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
2	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
3	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
4	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
5	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
6	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
7	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
8	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
9	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
10	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
11	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
12	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
13	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159
14	3360001	65879	8 975 535 44 70	Тула, наб. Горная 81	2025-03-01 18:40:35.811159	2025-03-01 21:41:35.811159	2025-03-01 19:17:35.811159	2025-03-01 21:41:35.811159

Total rows: 100 Query complete 00:00:00.049 LF Ln 2, Col 10

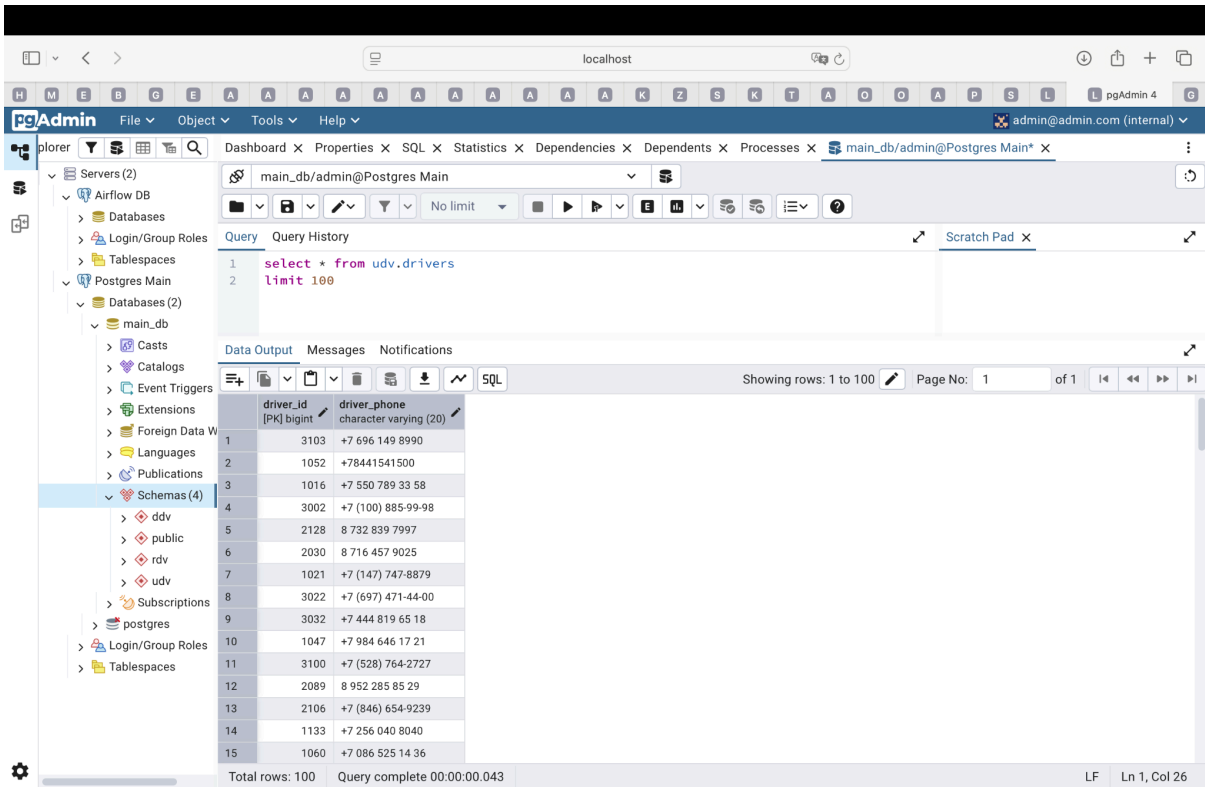
b. UDV

i. udv.drivers

We can identify in this table that there's many formats for the phone column, so it can be tuned and unified to unique format . For example

+7 (xxx) xxx-xx-xx

This column is not critical for this scenario, so we didn't include the data cleaning here.



The screenshot shows the pgAdmin 4 web interface. On the left, a tree view shows the database structure: Servers (2) > Airflow DB > Databases > Login/Group Roles > Tablespaces > Postgres Main > Databases (2) > main_db > Schemas (4) > udv. The main panel displays a SQL query: `select * from udv.drivers limit 100`. Below the query, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: `driver_id` (bigint) and `driver_phone` (character varying (20)). The table contains 15 rows of data. The status bar at the bottom indicates 'Total rows: 100' and 'Query complete 00:00:00.043'.

driver_id	driver_phone
3103	+7 696 149 8990
1052	+78441541500
1016	+7 550 789 33 58
3002	+7 (100) 885-99-98
2128	8 732 839 7997
2030	8 716 457 9025
1021	+7 (147) 747-8879
3022	+7 (697) 471-44-00
3032	+7 444 819 65 18
1047	+7 984 646 17 21
3100	+7 (528) 764-2727
2089	8 952 285 85 29
2106	+7 (846) 654-9239
1133	+7 256 040 8040
1060	+7 086 525 14 36

ii. udv.deliveries

pgAdmin 4 interface showing a query execution result for the `udv.deliveries` table. The query is:

```
select * from udv.deliveries
limit 100
```

The results show 100 rows of delivery data. A green status bar at the bottom indicates: "Successfully run. Total query runtime: 40 msec. 100 rows affected."

order_id	delivery_started_at	delivered_at	delivery_cost	driver_id
5962112	2025-04-05 15:08:53.601992	2025-04-05 16:09:53.601992	201.00	2119
3220946	2025-01-01 14:53:38.407082	2025-01-01 15:51:38.407082	359.00	1136
6142899	2025-10-25 15:33:09.874309	2025-10-25 16:34:09.874309	108.00	3086
6382732	2024-12-26 08:56:08.240622	[null]	160.00	2049
3324496	2025-08-11 23:01:26.784594	2025-08-12 00:08:26.784594	203.00	3075
3322625	2025-08-31 00:50:10.780333	2025-08-31 03:46:10.780333	299.00	1135
1600719	2025-06-05 15:50:02.084565	2025-06-05 17:35:02.084565	284.00	1104
6383651	2025-12-08 16:18:14.477071	2025-12-08 18:30:14.477071	184.00	1058
5963075	2024-12-19 19:12:26.278924	2024-12-19 20:48:26.278924	336.00	3029
3224510	[null]	[null]	315.00	2046
5863286	2025-04-18 15:26:18.742788	2025-04-18 16:15:18.742788	375.00	3056
3363747	2025-04-27 21:35:02.660448	2025-04-27 22:47:02.660448	129.00	1088
1604343	2025-08-23 07:42:56.982491	2025-08-23 08:52:56.982491	282.00	2009
5964119	[null]	[null]	214.00	1126
5863990	2025-08-16 03:30:32.321705	2025-08-16 03:59:32.321705	135.00	

iii. udv.items

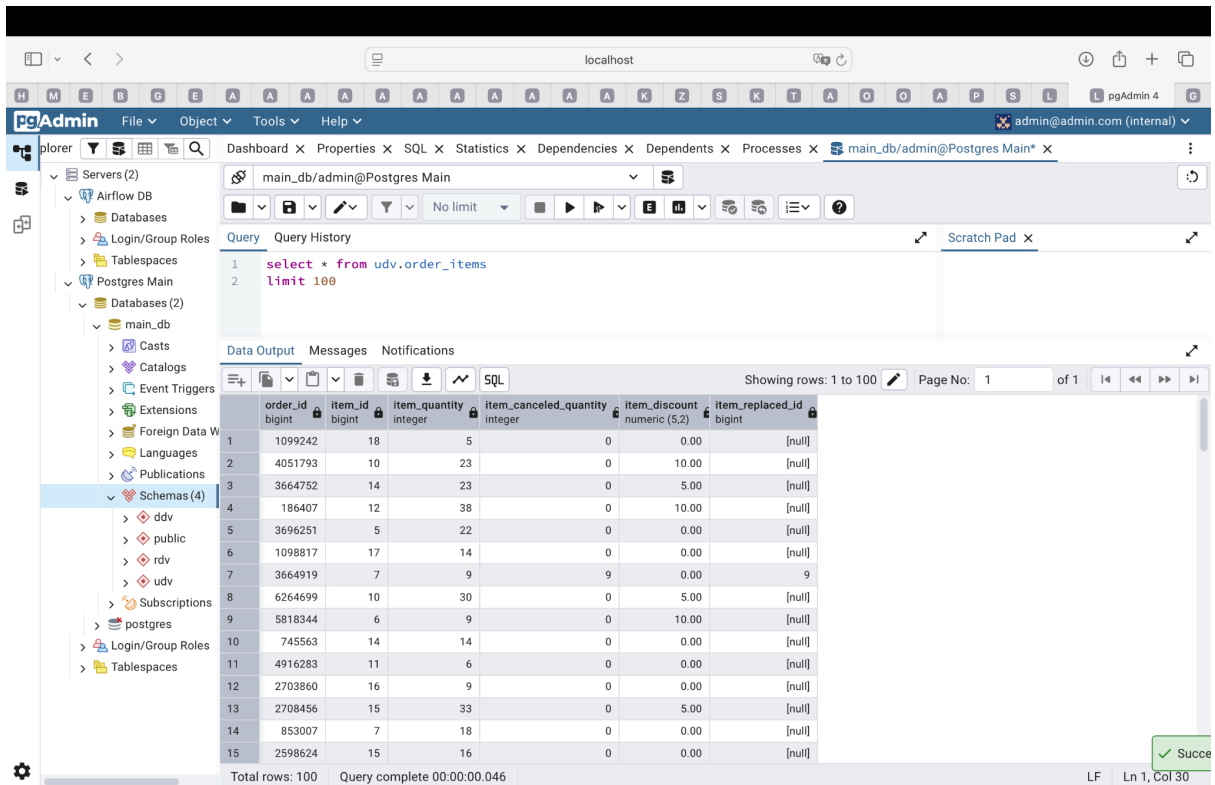
pgAdmin 4 interface showing a query execution result for the `udv.items` table. The query is:

```
select * from udv.items
limit 100
```

The results show 18 rows of item data. A green status bar at the bottom indicates: "Successfully run. Total query runtime: 39 msec. 18 rows affected."

item_id [PK]	item_title	item_price	item_category
5	Помидоры	103.00	Овощи
8	Кефир	243.00	Молочные продук...
7	Молоко	219.00	Молочные продук...
6	Капуста	580.00	Овощи
14	Говядина	567.00	Мясо
9	Творог	233.00	Молочные продук...
18	Булочка	129.00	Хлеб
17	Черный хлеб	332.00	Хлеб
3	Апельсины	99.00	Фрукты
15	Курица	471.00	Мясо
13	Свинина	209.00	Мясо
4	Огурцы	470.00	Овощи
16	Батон	584.00	Хлеб
12	Сок	109.00	Напитки
1	Яблоки	335.00	Фрукты

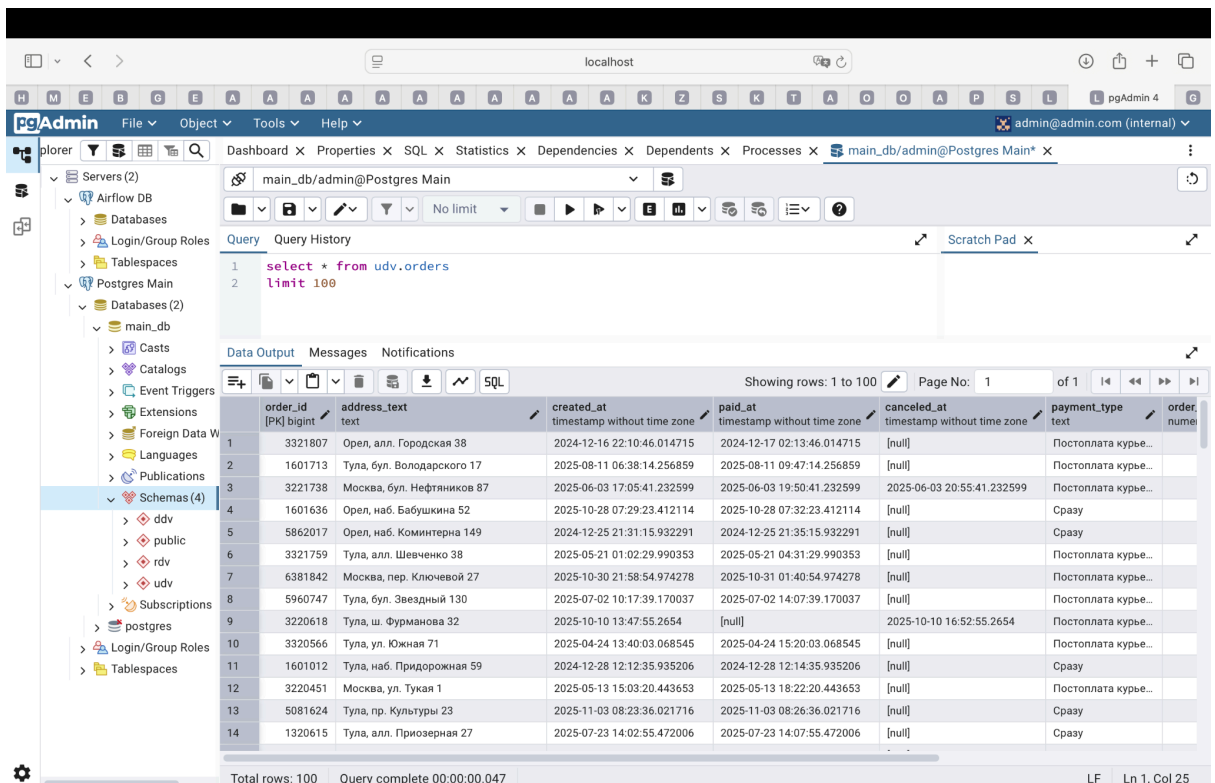
iv. udv.orders_items



The screenshot shows the pgAdmin 4 interface with a query executed against the `udv.orders_items` table. The query is `select * from udv.orders_items limit 100`. The result set contains 15 rows of data. The status bar at the bottom indicates 'Total rows: 100' and 'Query complete 00:00:00.046'.

	order_id bigint	item_id bigint	item_quantity integer	item_canceled_quantity integer	item_discount numeric (5,2)	item_replaced_id bigint
1	1099242	18	5	0	0.00	[null]
2	4051793	10	23	0	10.00	[null]
3	3664752	14	23	0	5.00	[null]
4	186407	12	38	0	10.00	[null]
5	3696251	5	22	0	0.00	[null]
6	1098817	17	14	0	0.00	[null]
7	3664919	7	9	9	0.00	9
8	6264699	10	30	0	5.00	[null]
9	5818344	6	9	0	10.00	[null]
10	745563	14	14	0	0.00	[null]
11	4916283	11	6	0	0.00	[null]
12	2703860	16	9	0	0.00	[null]
13	2708456	15	33	0	5.00	[null]
14	853007	7	18	0	0.00	[null]
15	2598624	15	16	0	0.00	[null]

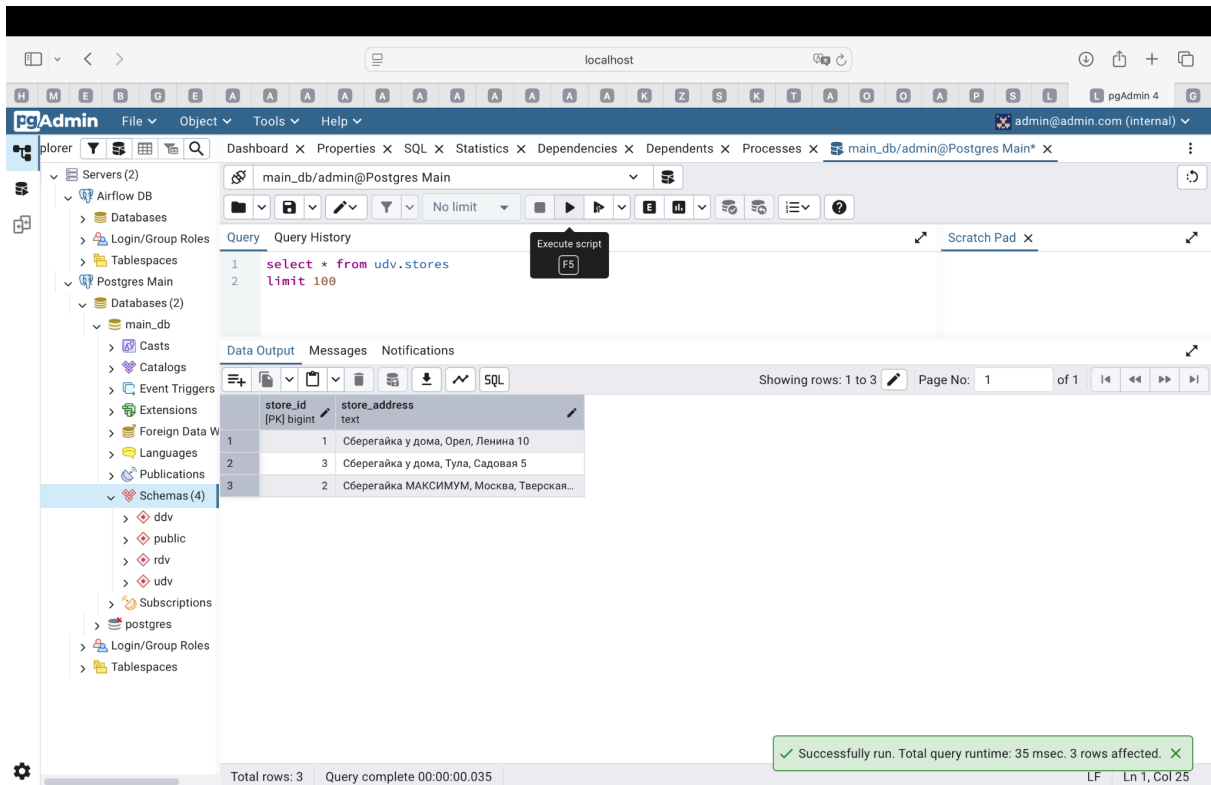
v. udv.orders



The screenshot shows the pgAdmin 4 interface with a query executed against the `udv.orders` table. The query is `select * from udv.orders limit 100`. The result set contains 14 rows of data. The status bar at the bottom indicates 'Total rows: 100' and 'Query complete 00:00:00.047'.

	order_id [PK] bigint	address_text text	created_at timestamp without time zone	paid_at timestamp without time zone	canceled_at timestamp without time zone	payment_type text	order_num
1	3321807	Орел, алл. Городская 38	2024-12-16 22:10:46.014715	2024-12-17 02:13:46.014715	[null]	Постоплата курье...	
2	1601713	Тула, бул. Володарского 17	2025-08-11 06:38:14.256859	2025-08-11 09:47:14.256859	[null]	Постоплата курье...	
3	3221738	Москва, бул. Нефтяников 87	2025-06-03 17:05:41.232599	2025-06-03 19:50:41.232599	2025-06-03 20:55:41.232599	Постоплата курье...	
4	1601636	Орел, наб. Бабушкина 52	2025-10-28 07:29:23.412114	2025-10-28 07:32:23.412114	[null]	Сразу	
5	5862017	Орел, наб. Коминтерна 149	2024-12-25 21:31:15.932291	2024-12-25 21:35:15.932291	[null]	Сразу	
6	3321759	Тула, алл. Шевченко 38	2025-05-21 01:02:29.990353	2025-05-21 04:31:29.990353	[null]	Постоплата курье...	
7	6381842	Москва, пер. Ключевой 27	2025-10-30 21:58:54.974278	2025-10-31 01:40:54.974278	[null]	Постоплата курье...	
8	5960747	Тула, бул. Звездный 130	2025-07-02 10:17:39.170037	2025-07-02 14:07:39.170037	[null]	Постоплата курье...	
9	3220618	Тула, ш. Фурманова 32	2025-10-10 13:47:55.2654	[null]	2025-10-10 16:52:55.2654	Постоплата курье...	
10	3320566	Тула, ул. Южная 71	2025-04-24 13:40:03.068545	2025-04-24 15:20:03.068545	[null]	Постоплата курье...	
11	1601012	Тула, наб. Придорожная 59	2024-12-28 12:12:35.935206	2024-12-28 12:14:35.935206	[null]	Сразу	
12	3220451	Москва, ул. Тукая 1	2025-05-13 15:03:20.443653	2025-05-13 18:22:20.443653	[null]	Постоплата курье...	
13	5081624	Тула, пр. Культуры 23	2025-11-03 08:23:36.021716	2025-11-03 08:26:36.021716	[null]	Сразу	
14	1320615	Тула, алл. Приозерная 27	2025-07-23 14:02:55.472006	2025-07-23 14:07:55.472006	[null]	Сразу	

vi. udv.stores



The screenshot shows the pgAdmin 4 web interface. The left sidebar displays the database structure, with the 'udv' schema selected under 'main_db'. The central pane shows a SQL query: `select * from udv.stores limit 100`. The 'Data Output' tab is active, displaying the results of the query in a table format. The table has two columns: 'store_id' (PK) and 'store_address'. The results show three rows of data. A status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.035'. A green notification box at the bottom right states 'Successfully run. Total query runtime: 35 msec. 3 rows affected.'

store_id [PK] bigint	store_address text
1	Сберегайка у дома, Орел, Ленина 10
2	Сберегайка у дома, Тула, Садовая 5
3	Сберегайка МАКСИМУМ, Москва, Тверская...

vii. udv.users

In this table, as we mentioned, we could perform a data cleaning for obtain an uniform phone number format.

pgAdmin

FileObjectToolsHelp

admin@admin.com (internal)

DashboardPropertiesSQLStatisticsDependenciesDependentsProcessesmain_db/admin@Postgres Main

main_db/admin@Postgres Main

No limit

QueryQuery HistoryScratch Pad

1select * from udv.users

2limit 100

Data OutputMessagesNotifications

Showing rows: 1 to 100Page No: 1 of 1

	user_id [PK] bigint	user_phone character varying (20)
1	247443	+7 158 055 71 08
2	113411	8 368 380 5419
3	190757	8 (570) 183-9939
4	317820	+7 764 503 2279
5	43606	+7 568 094 77 13
6	173806	82065656567
7	325311	8 561 950 16 20
8	152712	83028282508
9	215493	8 (945) 424-1012
10	65682	83735221613
11	104396	8 194 748 70 57
12	84968	+7 (954) 008-0597
13	11938	8 488 595 32 36
14	79342	8 (671) 122-44-80
15	175805	8 (427) 778-99-38

✓ Successfully run. Total query runtime: 37 msec. 100 rows affected. ✕

Total rows: 100Query complete 00:00:00.037LF Ln 1, Col 24

Servers (2)

- Airflow DB
 - Databases
 - Login/Group Roles
 - Tablespaces
- Postgres Main
 - Databases (2)
 - main_db
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data W
 - Languages
 - Publications
 - Schemas (4)
 - ddv
 - public
 - rdv
 - udv
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

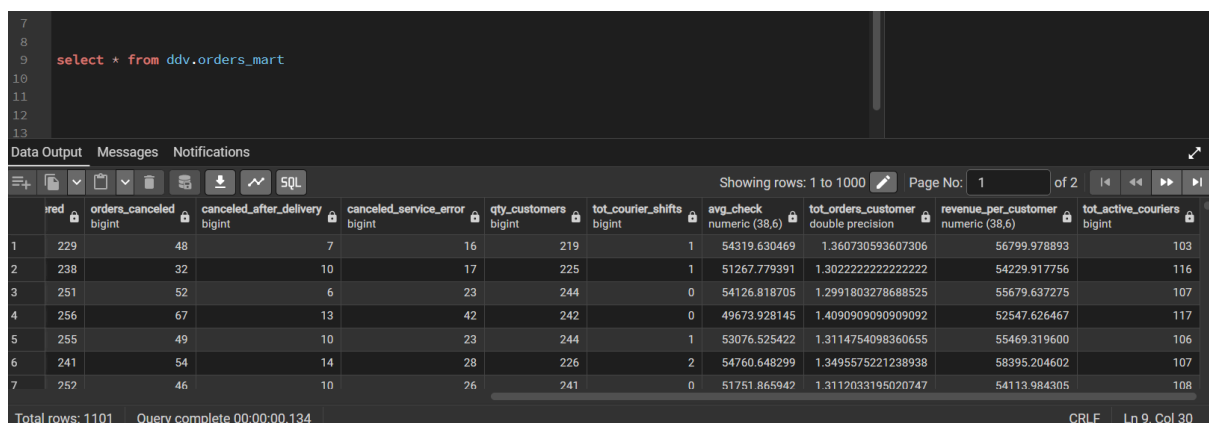
c. DDV

i. ddv.orders_mart

For this Datamart we have defined the following fields:

1. year: Year
2. month: Month
3. day: Day
4. city: City
5. store:store ID
6. turnover:Turnover (order amount including discounts)
7. revenue:Revenue (payment amount including discounts and substitutions)
8. profit: Profit (revenue - expenses)
9. orders_created: Number of orders placed (created)
- 10.orders_delivered: Number of delivered orders
- 11.orders_canceled: Number of canceled orders
- 12.canceled_after_delivery: Number of cancellations after delivery
- 13.canceled_service_error: Number of cancellations due to service errors (for cancellation reasons "Application error" and "Payment problems")
- 14.qty_customers: Number of customers
- 15.avg_check: Average check(revenue/orders delivered)
- 16.tot_orders_customer: Number of orders per customer
- 17.revenue_per_customer: Revenue per customer
- 18.tot_courier_shifts: Number of courier shifts (orders with shifts)
- 19.tot_active_couriers: Number of active couriers

For this datamart we have added additional filters for some of the metrics, regarding if the order was delivered successfully and not canceled.



The screenshot shows a SQL query editor with a query and its results. The query is:

```
select * from ddv.orders_mart
```

The results are displayed in a table with the following columns:

	orders_canceled bigint	canceled_after_delivery bigint	canceled_service_error bigint	qty_customers bigint	tot_courier_shifts bigint	avg_check numeric (38,6)	tot_orders_customer double precision	revenue_per_customer numeric (38,6)	tot_active_couriers bigint
1	229	48	7	16	219	1	54319.630469	1.360730593607306	56799.978893
2	238	32	10	17	225	1	51267.779391	1.3022222222222222	54229.917756
3	251	52	6	23	244	0	54126.818705	1.2991803278688525	55679.637275
4	256	67	13	42	242	0	49673.928145	1.4090909090909092	52547.626467
5	255	49	10	23	244	1	53076.525422	1.3114754098360655	55469.319600
6	241	54	14	28	226	2	54760.648299	1.3495575221238938	58395.204602
7	252	46	10	26	241	0	51751.865942	1.3112033195020747	54113.984305

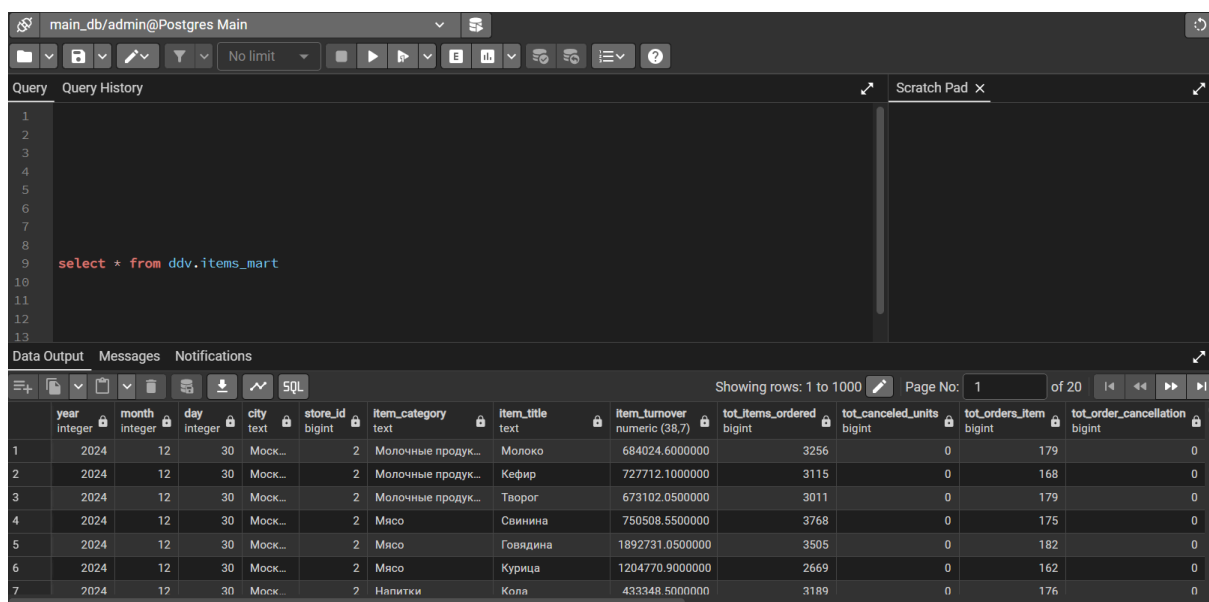
Total rows: 1101 | Query complete 00:00:00.134 | CRLF | Ln 9, Col 30

ii. ddv.items_mart

For this Datamart we have defined the following fields:

1. year:Year
2. month:Month
3. day:Day
4. city:City
5. store_id:Store
6. item_category:Category
7. item_id:Product
8. item_turnover:Product Turnover(this includes cancelations)
9. tot_items_ordered:Number of ordered units
- 10.tot_canceled_units:Number of canceled units
- 11.tot_orders_item: Number of orders with the product
- 12.tot_order_cancellation: Number of orders with cancellations
- 13.most_pop_item_per_day:Most popular product on the day
- 14.most_pop_item_per_day:Most popular product on the week
- 15.most_pop_item_per_day:Most popular product on the month
- 16.lst_pop_item_per_day:Least popular product on the day
- 17.lst_pop_item_per_week:Least popular product on the week
- 18.lst_pop_item_per_month:Least popular product on the month

For this datamart we have added an initial filter to only consider items contained in orders that were not canceled.



The screenshot shows a PostgreSQL query editor interface. The query editor at the top contains the following SQL query:

```
select * from ddv.items_mart
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 13 columns and 7 rows of data. The columns are: year, month, day, city, store_id, item_category, item_title, item_turnover, tot_items_ordered, tot_canceled_units, tot_orders_item, tot_order_cancellation, and item_id. The data is as follows:

	year	month	day	city	store_id	item_category	item_title	item_turnover	tot_items_ordered	tot_canceled_units	tot_orders_item	tot_order_cancellation	item_id
1	2024	12	30	Моск...	2	Молочные продук...	Молоко	684024.6000000	3256	0	179	0	
2	2024	12	30	Моск...	2	Молочные продук...	Кефир	727712.1000000	3115	0	168	0	
3	2024	12	30	Моск...	2	Молочные продук...	Творог	673102.0500000	3011	0	179	0	
4	2024	12	30	Моск...	2	Мясо	Свинина	750508.5500000	3768	0	175	0	
5	2024	12	30	Моск...	2	Мясо	Говядина	1892731.0500000	3505	0	182	0	
6	2024	12	30	Моск...	2	Мясо	Курица	1204770.9000000	2669	0	162	0	
7	2024	12	30	Моск...	2	Напитки	Кола	433348.5000000	3189	0	176	0	

5. Airflow

a. DAGS

DAGs										
All 8		Active 3	Paused 0	Running 2		Failed 0	Filter DAGs by tag	Search DAGs	Auto-refresh	
DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions		Lir	
00_create_schemas ddv rdv udv	airflow	1	None	2025-12-25, 20:40:49		1	▶ ⌂ 🗑		..	
01_parquet_to_rdv rdv	airflow	1	None	2025-12-25, 20:41:51		1	▶ ⌂ 🗑		..	
02_rdv_to_udv sql udv	airflow	1	None	2025-12-25, 20:44:54		1	▶ ⌂ 🗑		..	
03_udv_to_ddv_orders_mart ddv orders pyspark	airflow	1	None	2025-12-25, 20:47:57		1	▶ ⌂ 🗑		..	
04_udv_to_ddv_items_mart ddv items pyspark	airflow	1	None	2025-12-25, 20:47:56		1	▶ ⌂ 🗑		..	
05_master_pipeline master pipeline	airflow	1	None	2025-12-25, 20:40:47		1	▶ ⌂ 🗑		..	
11_test_postgres_connection postgres test	airflow	1	None	2025-12-25, 20:40:52		1	▶ ⌂ 🗑		..	
12_test_spark_connection	airflow	1	None	2025-12-25, 20:40:54		1	▶ ⌂ 🗑		..	

To run the entire workflow, it's only needed to run the DAG **05_master_pipeline**. We also have designed dag 11 and 12 for testing connectivity to postgresql and to check the installation of spark library, correspondingly.